

Pandas 中 read_csv 函数的 usecols 参数

作用

usecols 参数用于指定要读取的列，可以只读取 CSV 文件中的部分列，而不是全部列。这样可以：

节省内存：只加载需要的列

提高读取速度：减少 I/O 操作

简化数据处理：直接获取目标列，避免后续筛选

语法

代码块

```
1 pandas.read_csv(filepath, usecols=[列名或列索引列表])
```

使用方式

方式 1：通过列名指定

代码块

```
1 import pandas as pd
2
3 # 示例 CSV 文件内容 (data.csv):
4 # name,age,city,salary,department
5 # 张三,25,北京,8000,技术部
6 # 李四,30,上海,12000,市场部
7 # 王五,28,广州,9500,技术部
8
9 # 只读取 name 和 salary 两列
10 df = pd.read_csv('data.csv', usecols=['name', 'salary'])
11 print(df)
```

输出：

代码块

```
1      name  salary
2  0  张三    8000
3  1  李四   12000
```

```
4    2    王五    9500
```

方式 2：通过列索引指定

代码块

```
1  # 只读取第 0 列和第 3 列（索引从 0 开始）
2  df = pd.read_csv('data.csv', usecols=[0, 3])
3  print(df)
```

输出：

代码块

```
1      name  salary
2    0  张三    8000
3    1  李四   12000
4    2  王五    9500
```

方式 3：使用函数动态选择

代码块

```
1  # 只读取列名包含 'a' 的列
2  df = pd.read_csv('data.csv', usecols=lambda x: 'a' in x)
3  print(df)
```

输出：

代码块

```
1      name  salary  department
2    0  张三    8000      技术部
3    1  李四   12000      市场部
4    2  王五    9500      技术部
```

实际应用场景

场景 1：大文件只读取部分列

```
代码块 假设有一个包含 100 列的大型 CSV 文件
2 # 只需要其中 3 列进行分析
3 df = pd.read_csv('large_dataset.csv',
4                   usecols=['user_id', 'purchase_amount', 'date'])
```

场景 2：配合其他参数使用

代码块

```
1 # 只读取特定列，并指定数据类型
2 df = pd.read_csv('data.csv',
3                   usecols=['name', 'age', 'salary'],
4                   dtype={'age': int, 'salary': float})
```

场景 3：读取不连续的列

代码块

```
1 # 读取第 1、3、5 列
2 df = pd.read_csv('data.csv', usecols=[0, 2, 4])
```

注意事项

列名必须存在：如果使用列名列表，列名必须在 CSV 文件中存在，否则会报错

索引从 0 开始：使用列索引时，从 0 开始计数

不能混用：usecols 要么是列名列表，要么是列索引列表，不能混合使用

顺序保持：读取的列会按照你指定的顺序排列

性能对比示例

代码块

```
1 import pandas as pd
2 import time
3
4 # 假设有一个 100MB 的 CSV 文件，包含 50 列
5
6 # 方式 1：读取所有列
7 start = time.time()
8 df_all = pd.read_csv('large_file.csv')
9 print(f"读取所有列耗时: {time.time() - start:.2f} 秒")
```

```
10 print(f"内存占用: {df_all.memory_usage().sum() / 1024 / 1024:.2f} MB")
11
12 # 方式 2: 只读取需要的 5 列
13 start = time.time()
14 df_subset = pd.read_csv('large_file.csv',
15                          usecols=['col1', 'col5', 'col10', 'col20', 'col30'])
16 print(f"只读取 5 列耗时: {time.time() - start:.2f} 秒")
17 print(f"内存占用: {df_subset.memory_usage().sum() / 1024 / 1024:.2f} MB")
```

典型结果：使用 usecols 可以节省 80-90% 的内存和读取时间。

总结

usecols 是 Pandas 读取 CSV 文件时非常实用的参数，特别是在处理大型数据集时，合理使用可以显著提升性能并节省系统资源。